Mini Project 2: Creator Dashboard Lite

Console Application for Digital Creators with Persistent File Handling in C

Project Architecture & Overview

The **Creator Dashboard Lite** is a complete console application designed to manage digital content ideas using standard C. It bridges foundational object-oriented thinking with procedural persistence techniques, implementing structures, file streams (`fopen`, `fprintf`, `fgets`), string tokenization (`strtok`), and CRUD operations.

- **Data Encapsulation:** Using the `Content` struct to bundle title, platform, views, and status.
- **Delimiter-Based Storage:** Utilizing the pipe (`|`) separator in `content_list.txt` to safely handle strings with spaces or commas.
- **Load-Modify-Overwrite Pattern:** Managing updates and deletions by loading file records into an array, modifying them in memory, and rewriting the persistent storage.

Complete Source Code

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define FILE_NAME "content_list.txt"

/* Task 1: Content struct definition */
typedef struct {
    char title[50];
    char platform[20];
    int views;
    char status[15];
} Content;

void displayContent(Content *c) {
    printf("Title    : %s\n", c->title);
    printf("Platform : %s\n", c->platform);
    printf("Views    : %d\n", c->views);
    printf("Status   : %s\n", c->status);
    printf("-------------------------\n");
}

/* File Helpers: Load all records into array */
int loadAll(Content arr[]) {
    FILE *fp = fopen(FILE_NAME, "r");
    if (fp == NULL) return 0;
```

```
    char line[150];
    int n = 0;
    while (fgets(line, sizeof(line), fp) != NULL && n < 100) {
        line[strcspn(line, "\n")] = '\0';
        char *t = strtok(line, "|");
        char *p = strtok(NULL, "|");
        char *v = strtok(NULL, "|");
        char *s = strtok(NULL, "|");
        if (t && p && v && s) {
            strcpy(arr[n].title, t);
            strcpy(arr[n].platform, p);
            arr[n].views = atoi(v);
            strcpy(arr[n].status, s);
            n++;
        }
    }
    fclose(fp);
    return n;
}

/* Save all records back to file (overwrite) */
void saveAll(Content arr[], int n) {
    FILE *fp = fopen(FILE_NAME, "w");
    for (int i = 0; i < n; i++)
        fprintf(fp, "%s|%s|%d|%s\n",
                arr[i].title, arr[i].platform, arr[i].views, arr[i].status)
                    ;
    fclose(fp);
}

/* Task 2: Add content (append mode) */
void addContent() {
    Content c;
    char buffer[20];

    printf("\nEnter title    : ");
    fgets(c.title, sizeof(c.title), stdin);
    c.title[strcspn(c.title, "\n")] = '\0';

    printf("Enter platform : ");
    fgets(c.platform, sizeof(c.platform), stdin);
    c.platform[strcspn(c.platform, "\n")] = '\0';

    printf("Enter views    : ");
    fgets(buffer, sizeof(buffer), stdin);
    c.views = atoi(buffer);

    printf("Enter status   : ");
    fgets(c.status, sizeof(c.status), stdin);
    c.status[strcspn(c.status, "\n")] = '\0';

    FILE *fp = fopen(FILE_NAME, "a");
    if (fp == NULL) { printf("Error opening file!\n"); return; }
    fprintf(fp, "%s|%s|%d|%s\n", c.title, c.platform, c.views, c.status);
    fclose(fp);

    printf("\n[OK] Content added successfully!\n");
}
```

```
/* Task 3: View all content */
void viewContent() {
    Content arr[100];
    int n = loadAll(arr);

    printf("\n--- Content Ideas ---\n");
    if (n == 0) { printf("(No content ideas yet!)\n"); return; }
    for (int i = 0; i < n; i++)
        printf("%d. %-30s [%s]\n", i + 1, arr[i].title, arr[i].platform);
}

/* Task 4: Update status */
void updateStatus() {
    Content arr[100];
    int n = loadAll(arr);
    if (n == 0) { printf("\nNothing to update!\n"); return; }

    viewContent();
    int num;
    printf("\nEnter the number to update: ");
    scanf("%d", &num); getchar();
    if (num < 1 || num > n) { printf("Invalid number!\n"); return; }

    printf("Current status: %s\n", arr[num - 1].status);
    printf("Enter new status (Idea/Draft/Published): ");
    fgets(arr[num - 1].status, sizeof(arr[num - 1].status), stdin);
    arr[num - 1].status[strcspn(arr[num - 1].status, "\n")] = '\0';

    saveAll(arr, n);
    printf("\n[OK] Status updated successfully!\n");
}

/* Task 5: Delete content */
void deleteContent() {
    Content arr[100];
    int n = loadAll(arr);
    if (n == 0) { printf("\nNothing to delete!\n"); return; }

    viewContent();
    int num;
    printf("\nEnter the number to DELETE: ");
    scanf("%d", &num); getchar();
    if (num < 1 || num > n) { printf("Invalid number!\n"); return; }

    printf("\nDeleted: %s\n", arr[num - 1].title);

    for (int i = num - 1; i < n - 1; i++)
        arr[i] = arr[i + 1];
    n--;

    saveAll(arr, n);

    printf("\n--- Updated List ---\n");
    if (n == 0) printf("(List is now empty)\n");
    for (int i = 0; i < n; i++)
        printf("%d. %-30s [%s]\n", i + 1, arr[i].title, arr[i].platform);
}
```

```
int main() {
    int choice;
    printf("============================================\n");
    printf("     CREATOR DASHBOARD LITE  v1.0\n");
    printf("============================================\n");

    while (1) {
        printf("\n========== MAIN MENU ==========\n");
        printf("1. Add new content idea\n");
        printf("2. View all content ideas\n");
        printf("3. Update status of an idea\n");
        printf("4. Delete a content idea\n");
        printf("5. View full details of idea #1\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");

        scanf("%d", &choice);
        getchar();

        switch (choice) {
            case 1: addContent();    break;
            case 2: viewContent();   break;
            case 3: updateStatus();  break;
            case 4: deleteContent(); break;
            case 5: {
                Content arr[100];
                int n = loadAll(arr);
                if (n > 0) displayContent(&arr[0]);
                else printf("\nNo items to display.\n");
                break;
            }
            case 6:
                printf("\nGoodbye, creator! Keep creating!\n");
                return 0;
            default:
                printf("\nInvalid choice! Try again.\n");
        }
    }
}
```

Task Breakdown Summary

#	Feature	File Mode	Core Mechanism
1	Content Struct	None (In-Memory)	4-member layout definition
2	Add Content	`"a"` (Append)	`fprintf()` to end of file
3	View Content	`"r"` (Read)	`fgets()` + `strtok()` parser
4	Update Status	`"w"` (Overwrite)	Load → Modify → Save
5	Delete Item	`"w"` (Overwrite)	Array shift left → Rewrite

Key Takeaway: Combining safe string parsing (‘strtok‘) with robust array manipulation patterns allows lightweight C programs to manage persistent data reliably without external database dependencies.